

CO3-AT3-Part3

DATA INTERPRETATION

#=====

PPO,TRPO & DDPG Performance Analysis

NAME:N.Akshaya

REG.NO:192425129

COURSE CODE:MLA0305

COURSE NAME: REINFORCEMENT LEARNING

SLOT:B

Python code:

```
import numpy as np

import matplotlib.pyplot as plt

# DATA INTERPRETATION – PPO, TRPO & DDPG PERFORMANCE ANALYSIS

np.random.seed(42)

# 1. Generate Performance Data

episodes=np.arange(1,101)

ppo_reward=50+180*(1-np.exp(-episodes/25))

trpo_reward=45+165*(1-np.exp(-episodes/30))

ddpg_reward=40+150*(1-np.exp(-episodes/35))

ppo_reward+=np.random.normal(0,5,100)
```

```

trpo_reward+=np.random.normal(0,7,100)

ddpg_reward+=np.random.normal(0,9,100)

ppo_reward=np.maximum(ppo_reward,0)

trpo_reward=np.maximum(trpo_reward,0)

ddpg_reward=np.maximum(ddpg_reward,0)

# 2. Performance Metrics

algorithms=["PPO","TRPO","DDPG"]

avg_reward=[np.mean(ppo_reward),np.mean(trpo_reward),np.mean(ddpg_reward)]

final_reward=[ppo_reward[-1],trpo_reward[-1],ddpg_reward[-1]]

success_rate=[92,86,80]

training_time=[18,25,32]

stability=[100-np.std(ppo_reward),100-np.std(trpo_reward),100-np.std(ddpg_reward)]

min_stability=min(stability)

max_stability=max(stability)

stability_normalized=[70+30*(x-min_stability)/(max_stability-min_stability) for x in
stability]

# 3. Summary Output

print("="*65)

print("DATA INTERPRETATION – PPO, TRPO & DDPG")

print("PERFORMANCE ANALYSIS")

print("="*65)

print("\nPERFORMANCE SUMMARY")

print("-"*65)

print(f'{"Algorithm":<12}{"Avg Reward":<15}{"Final\nReward":<15}{"Success %":<12}{"Training Time"}')

```

```

print("-"*65)

for i in range(3):

    print(f"{algorithms[i]:<12}{avg_reward[i]:<15.2f}{final_reward[i]:<15.2f}{success_rate[
i]:<12.2f}{training_time[i]} min")

print("-"*65)

# 4. Identify Best Algorithm

best_reward_index=np.argmax(avg_reward)

best_success_index=np.argmax(success_rate)

fastest_index=np.argmin(training_time)

print("\nINTERPRETATION")

print("-"*65)

print(f"Highest Average Reward : {algorithms[best_reward_index]}")

print(f"Highest Success Rate : {algorithms[best_success_index]}")

print(f"Fastest Training : {algorithms[fastest_index]}")

print("\nOverall Performance:")

print("PPO : Best overall learning performance and high stability.")

print("TRPO : Good performance with stable policy updates.")

print("DDPG : Useful for continuous control but slower and less stable.")

# 5. Visualization 1 – Learning Curve

plt.figure(figsize=(10,6))

plt.plot(episodes,ppo_reward,color="blue",linewidth=2.5,label="PPO")

plt.plot(episodes,trpo_reward,color="green",linewidth=2.5,label="TRPO")

plt.plot(episodes,ddpg_reward,color="red",linewidth=2.5,label="DDPG")

plt.title("PPO vs TRPO vs DDPG – Learning Curve")

```

```
plt.xlabel("Episodes")
plt.ylabel("Reward")
plt.grid(True,alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

6. Visualization 2 – Average Reward Comparison

```
plt.figure(figsize=(8,6))

bars=plt.bar(algorithms,avg_reward,color=["blue","green","red"],edgecolor="black")

plt.title("Average Reward Comparison")

plt.xlabel("Algorithm")

plt.ylabel("Average Reward")

for bar,value in zip(bars,avg_reward):

plt.text(bar.get_x()+bar.get_width()/2,bar.get_height()+2,f"{value:.2f}",ha="center",font
weight="bold")

plt.grid(axis="y",alpha=0.3)

plt.tight_layout()

plt.show()
```

7. Visualization 3 – Success Rate

```
plt.figure(figsize=(8,6))

bars=plt.bar(algorithms,success_rate,color=["blue","green","red"],edgecolor="black")

plt.title("Success Rate Comparison")

plt.xlabel("Algorithm")

plt.ylabel("Success Rate (%)")
```

```
plt.ylim(0,100)

for bar,value in zip(bars,success_rate):
    plt.text(bar.get_x()+bar.get_width()/2,bar.get_height()+2,f"{value}%",ha="center",fontw
eight="bold")

plt.grid(axis="y",alpha=0.3)

plt.tight_layout()

plt.show()
```

8. Visualization 4 – Training Time

```
plt.figure(figsize=(8,6))

bars=plt.bar(algorithms,training_time,color=["blue","green","red"],edgecolor="black")

plt.title("Training Time Comparison")

plt.xlabel("Algorithm")

plt.ylabel("Training Time (minutes)")

for bar,value in zip(bars,training_time):

    plt.text(bar.get_x()+bar.get_width()/2,bar.get_height()+0.5,f"{value}
min",ha="center",fontweight="bold")

plt.grid(axis="y",alpha=0.3)

plt.tight_layout()

plt.show()
```

9. Visualization 5 – Stability Comparison

```
plt.figure(figsize=(8,6))

bars=plt.bar(algorithms,stability_normalized,color=["blue","green","red"],edgecolor="bl
ack")

plt.title("Training Stability Comparison")

plt.xlabel("Algorithm")
```

```

plt.ylabel("Stability Score")

for bar,value in zip(bars,stability_normalized):

plt.text(bar.get_x()+bar.get_width()/2,bar.get_height()+0.5,f"{value:.2f}",ha="center",fontweight="bold")

plt.grid(axis="y",alpha=0.3)

plt.tight_layout()

plt.show()

```

10. Visualization 6 – Overall Performance

```

reward_score=np.array(avg_reward)

reward_score=((reward_score-reward_score.min()/(reward_score.max()-
reward_score.min()))*100

success_score=np.array(success_rate)

time_score=((max(training_time)-np.array(training_time))/(max(training_time)-
min(training_time)))*100

stability_normalized=np.array(stability_normalized)

overall_score=0.45*reward_score+0.30*success_score+0.15*stability_normalized+0.10*
time_score

plt.figure(figsize=(8,6))

bars=plt.bar(algorithms,overall_score,color=["blue","green","red"],edgecolor="black")

plt.title("Overall PPO vs TRPO vs DDPG Performance")

plt.xlabel("Algorithm")

plt.ylabel("Overall Performance Score")

plt.ylim(0,110)

for bar,value in zip(bars,overall_score):

plt.text(bar.get_x()+bar.get_width()/2,bar.get_height()+2,f"{value:.2f}",ha="center",fontweight="bold")

```

```

plt.grid(axis="y",alpha=0.3)

plt.tight_layout()

plt.show()

# 11. Final Result

best_overall=algorithms[np.argmax(overall_score)]

print("\n"+"="*65)

print("FINAL RESULT")

print("="*65)

print(f"Best Overall Algorithm : {best_overall}")

print("\nPPO -> High reward, high success rate and good stability")

print("TRPO -> Stable policy optimization with moderate performance")

print("DDPG -> Good for continuous control but comparatively slower")

print("\nConclusion:")

print("PPO provides the best overall performance among PPO, TRPO and DDPG for the
simulated experimental data.")

print("="*65)

```

Summary Output:

```

=====
          DATA INTERPRETATION – PPO, TRPO & DDPG
          PERFORMANCE ANALYSIS
=====

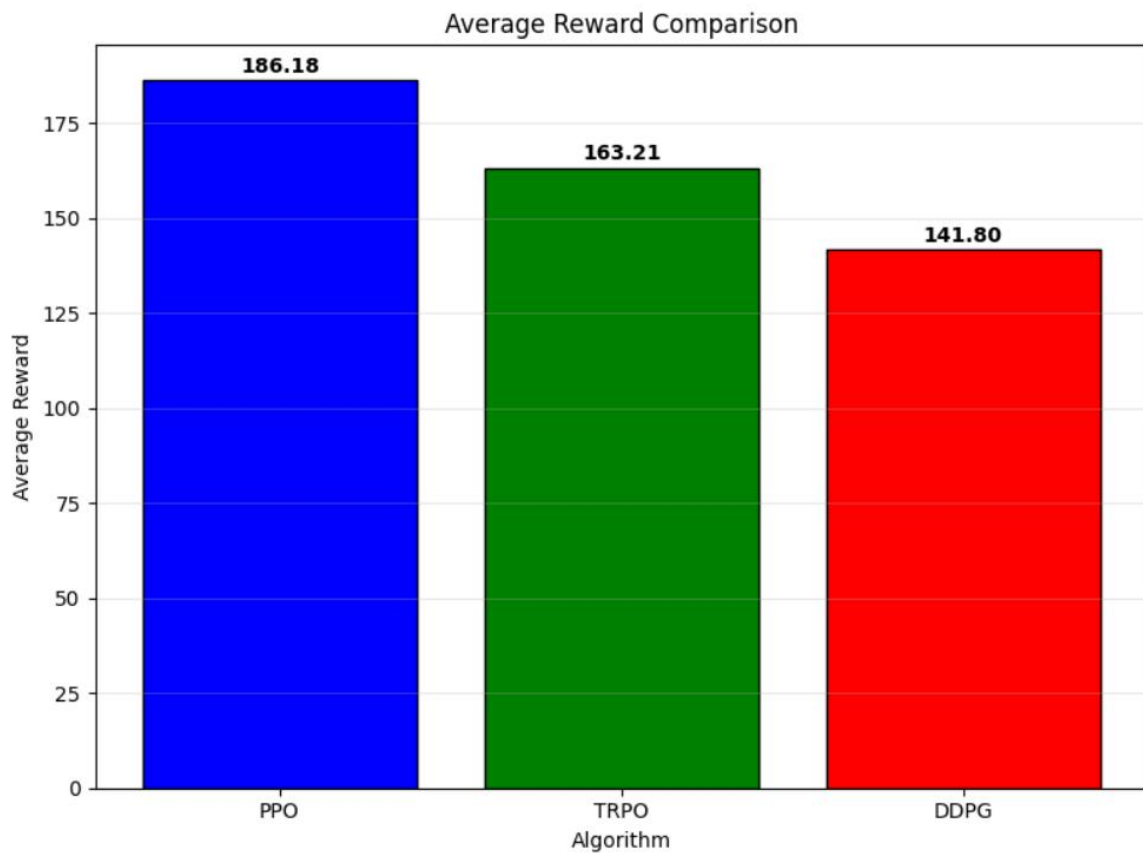
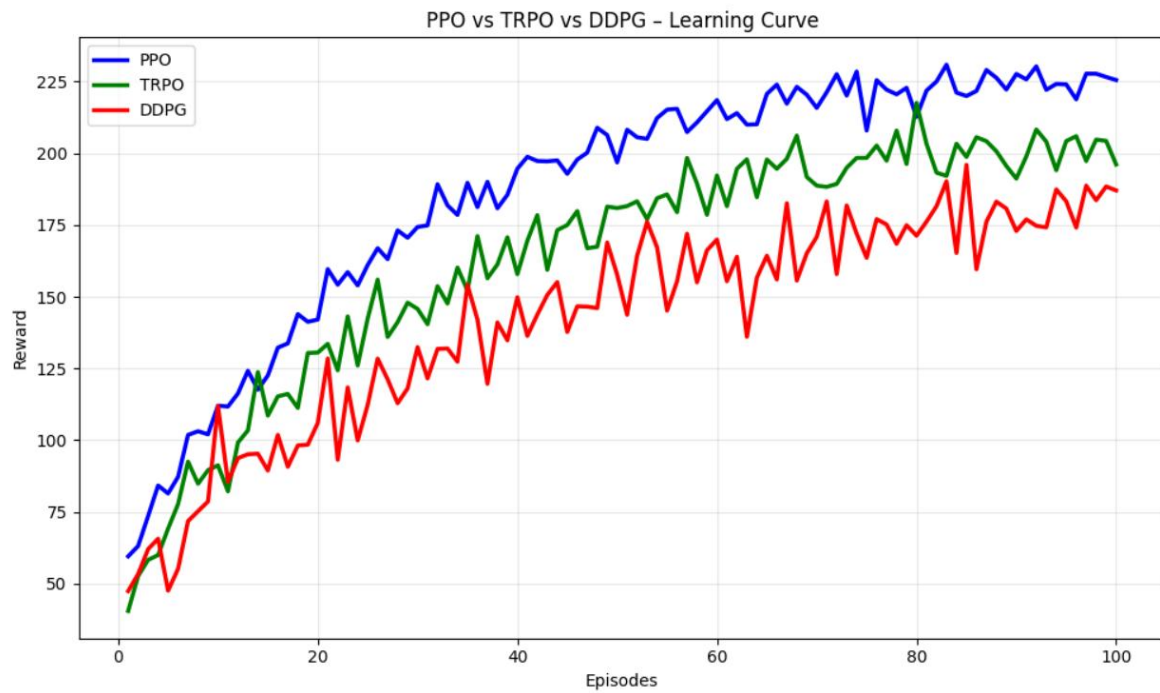
PERFORMANCE SUMMARY
-----
Algorithm    Avg Reward    Final Reward    Success %    Training Time
-----
PPO           186.18         225.53          92.00        18 min
TRPO          163.21         196.11          86.00        25 min
DDPG          141.80         187.05          80.00        32 min
-----

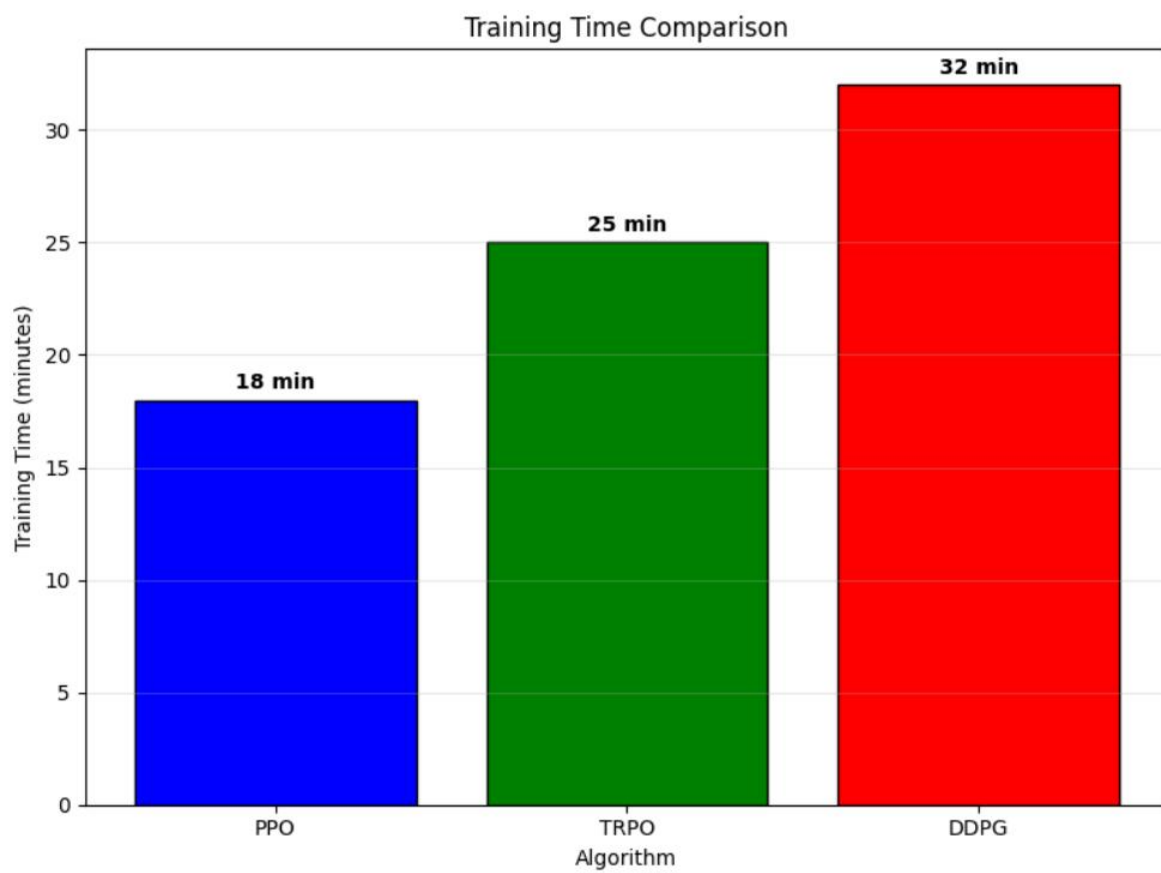
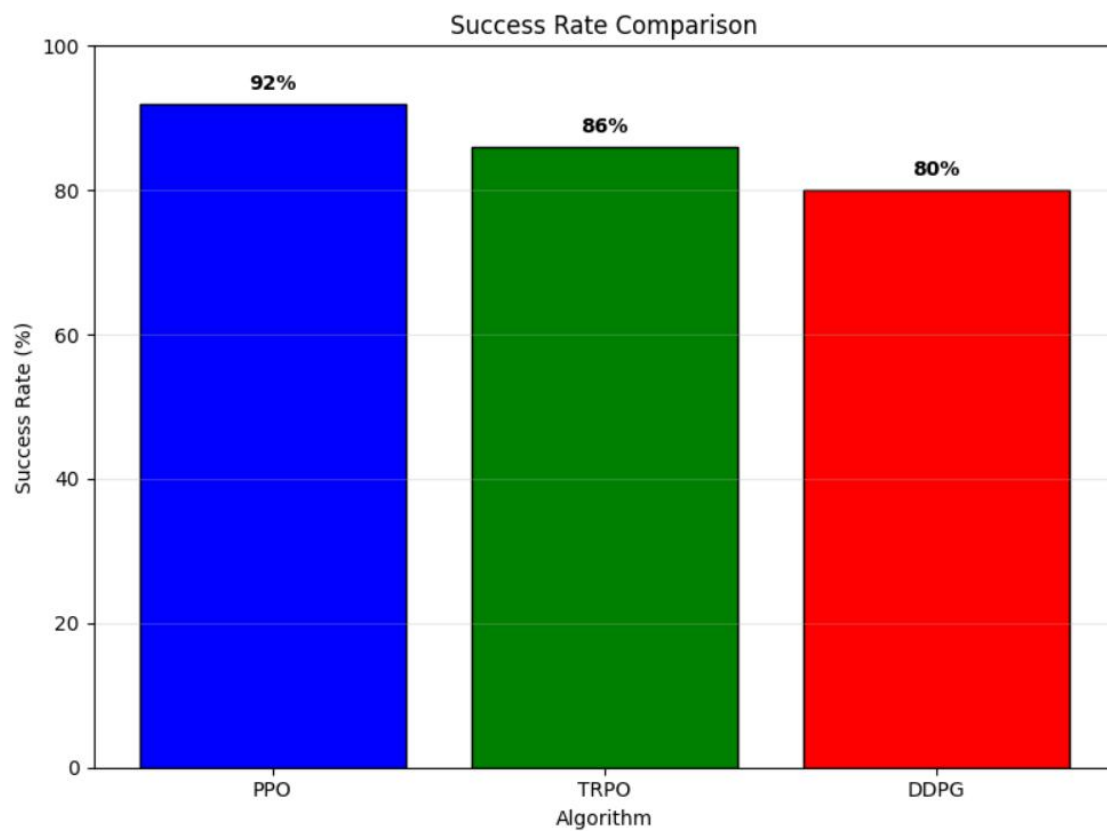
INTERPRETATION
-----
Highest Average Reward : PPO
Highest Success Rate   : PPO
Fastest Training       : PPO

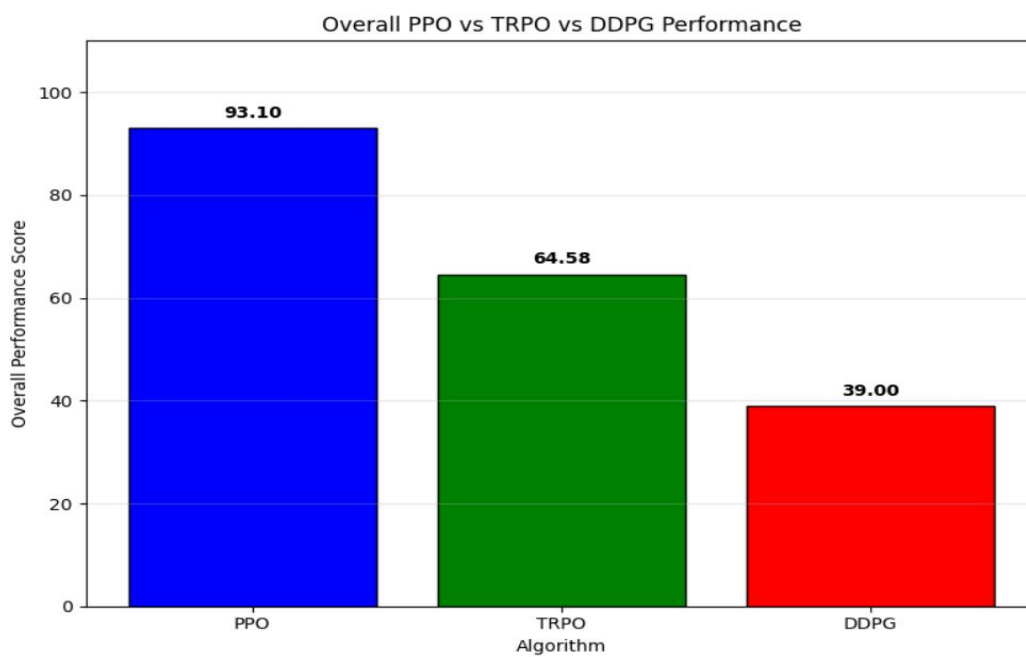
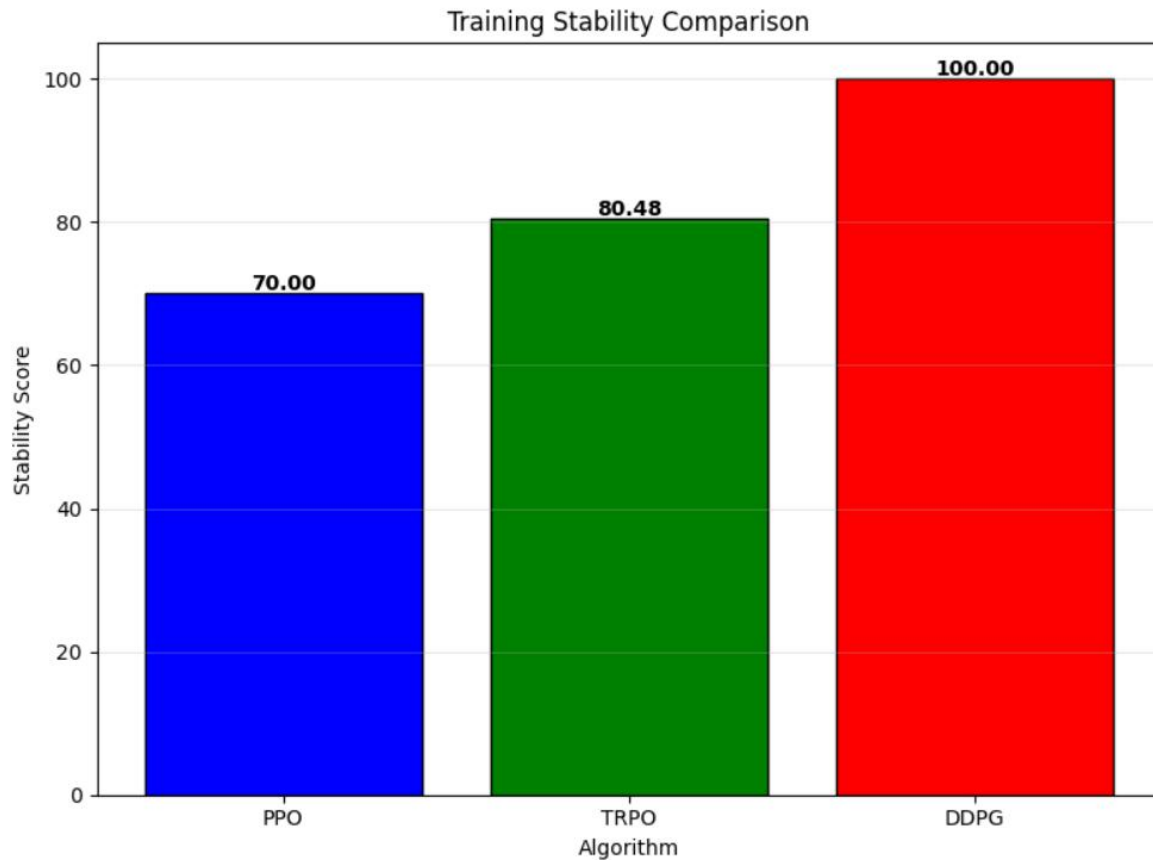
Overall Performance:
PPO : Best overall learning performance and high stability.
TRPO : Good performance with stable policy updates.
DDPG : Useful for continuous control but slower and less stable.

```

Visualization Output:







```
=====
                        FINAL RESULT
=====
Best Overall Algorithm : PPO

PPO -> High reward, high success rate and good stability
TRPO -> Stable policy optimization with moderate performance
DDPG -> Good for continuous control but comparatively slower

Conclusion:
PPO provides the best overall performance among PPO, TRPO and DDPG for the simulated experimental data.
=====
```